

Medicare Project Documentation

Comprehensive Full-Stack Hospital & Clinic Management System Architecture

1. Project Overview

Medicare is an enterprise-grade full-stack web application engineered to digitize and streamline modern hospital and clinical workflows. It provides tailored experiences for three key user tiers: **Patients**, **Doctors**, and **Administrators**.

The system is split into two distinct, decoupled codebases ensuring modularity and scalable deployment:

- **Backend** (`medicare-backend`): A robust Spring Boot REST API responsible for secure persistence, authentication, business logic, and AI assistant capabilities.
- **Frontend** (`medicare-frontend`): A responsive React single-page application built via Vite, utilizing role-based protected routing and dynamic modular views.

2. Technology Stack Architecture

Backend Ecosystem

- Framework** Spring Boot
- Build Tool** Maven Wrapper (`mvnw`)
- Security** Spring Security & JWT Filters
- Persistence** Spring Data JPA / Repositories

Frontend Ecosystem

- Bundler** Vite & React
- Routing** React Context & Protected Routes
- HTTP Client** Axios with Interceptors
- Styling** Modular CSS & Responsive Layouts

3. Database & Entity Schema

The relational database schema is structured around core operational entities:

- **User / Role**: Manages credentials, authorization tiers, and user profiles.
- **Doctor**: Extends user profiles with specialization credentials, departments, and scheduling metadata.
- **Department**: Categorizes hospital operational units.
- **Appointment**: Tracks booking schedules, timestamps, and progress statuses (`AppointmentStatus`).
- **Notification**: Stores system alerts and communication logs.

4. Backend API Controllers & Services

The backend architecture exposes specialized controllers mapping to domain requirements:

Controller	Key Responsibilities
AuthController	User registration, login, token generation, and credential exchanges.
AdminController	Administrative analytics, system oversight, and dashboard data aggregation.
DoctorController	Doctor scheduling, profile management, and doctor-specific KPIs.
AppointmentController	Creation, tracking, historical archiving, and status updates of appointments.
AIController	Interfaces with AI integrations to deliver intelligent health assistance tools.

5. Frontend Portal Modules

The user interface is separated into three dedicated operational dashboards:

- **Patient Portal:** Enables users to browse doctors, book and manage upcoming appointments, review appointment histories, adjust profile settings, and interact with the AI assistant.
- **Doctor Portal:** Empowers physicians to review daily schedules, manage patient treatment details, and update professional availability or passwords.
- **Admin Portal:** Provides comprehensive tools for managing departments, overseeing doctor and patient directories, tracking platform-wide appointments, and broadcasting system alerts.

6. Security & Exception Handling

Security is enforced via stateless **JSON Web Tokens (JWT)** processed through custom security filters (`JwtAuthenticationFilter`). Incoming requests are rigorously checked against role permissions defined in `SecurityConfig`.

Globally, exceptions are intercepted and managed uniformly via `GlobalExceptionHandler`, guaranteeing clean HTTP status codes and standardized error payloads across all client interactions.